ADVANCED SQL QUERY SYSTEM

Objectives

1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

1. Subqueries and Nested Queries

Definition:

A subquery is a query written inside another SQL query. The inner query is executed first, and its result is used by the outer query.

Types of Subqueries:

1. Single-row: Returns one value.
2. Multiple-row: Returns multiple values.
3. Nested: A query inside another query.
4. Correlated: Inner query depends on outer query.

Example: Find orders whose amount is greater than the average order amount.

```
SQL> SELECT *
  2  FROM SAVANA_ORDERS
  3  WHERE TOTAL_AMOUNT >
  4  (SELECT AVG(TOTAL_AMOUNT)
  5   FROM SAVANA_ORDERS);

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ ----------------------
------
      1001         101 22-SEP-26        26998 Confirmed
      9001         101 23-OCT-26        15999 Confirmed
```

2. Find Products Above Average Price

A subquery can be used to compare each product's price with the average product price.

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE
  2  FROM SHOP_PRODUCT
  3  WHERE PRICE >
  4  (SELECT AVG(PRICE)
  5   FROM SHOP_PRODUCT);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       101
Wireless Headphones
      1799

       105
Electric Kettle
      1299

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------

       106
Bluetooth Speaker
      1599

       109
Handbag

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
      1299

       110
Electric Kettle
      1199
```

```
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
Smart Watch
      2999

       104
Denim Jeans
      1499


7 rows selected.
```

2.1 Products Below Average Price

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, PRICE
  2  FROM SHOP_PRODUCT
  3  WHERE PRICE <
  4  (SELECT AVG(PRICE)
  5   FROM SHOP_PRODUCT);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       107
Casual Shirt
       799

       108
Face Wash
       499

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------

       901
Pen
       150

       902
Book

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       250

       903
Pencil
       100

       103
```

```
PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
Cotton Shirt
       899

       201
T-Shirt
       999


7 rows selected.
```

3. Identify Customers with Maximum Purchases

We can use SUM() along with a subquery to identify the customer who has the highest total purchase amount.

Step 1: Calculate Customer-wise Purchase

```
SQL> SELECT CUSTOMER_ID,
  2         SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  3  FROM SAVANA_ORDERS
  4  GROUP BY CUSTOMER_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        101          42997
        102           1999
        103           1598
```

Step 2: Find the Maximum Purchase

```
SQL> SELECT CUSTOMER_ID,
  2         SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  3  FROM SAVANA_ORDERS
  4  GROUP BY CUSTOMER_ID
  5  HAVING SUM(TOTAL_AMOUNT) =
  6  (
  7      SELECT MAX(TOTAL_PURCHASE)
  8      FROM
  9      (
 10          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
 11          FROM SAVANA_ORDERS
 12          GROUP BY CUSTOMER_ID
 13      )
 14  );

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        101          42997
```

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT CUSTOMER_ID,
  2         SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  3  FROM SAVANA_ORDERS
  4  GROUP BY CUSTOMER_ID
  5  HAVING SUM(TOTAL_AMOUNT) =
  6  (
  7      SELECT MAX(TOTAL_PURCHASE)
  8      FROM
  9      (
 10          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
 11          FROM SAVANA_ORDERS
 12          GROUP BY CUSTOMER_ID
 13      )
 14  );

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        101          42997
```

4. Complex Business Queries

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

4.1 Customers Who Placed Orders Above Average Order Value

```
SQL> SELECT CUSTOMER_ID, ORDER_ID, TOTAL_AMOUNT
  2  FROM SAVANA_ORDERS
  3  WHERE TOTAL_AMOUNT >
  4  (SELECT AVG(TOTAL_AMOUNT)
  5   FROM SAVANA_ORDERS);

CUSTOMER_ID   ORDER_ID TOTAL_AMOUNT
----------- ---------- ------------
        101       1001        26998
        101       9001        15999
```

5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated from ORDER_DETAILS.

```
SQL> SELECT PRODUCT_ID,
  2         SUM(SUBTOTAL) AS TOTAL_SALES
  3  FROM SAVANA_ORDER_DETAILS
  4  GROUP BY PRODUCT_ID
  5  HAVING SUM(SUBTOTAL) >
  6  (
  7      SELECT AVG(PRODUCT_SALES)
  8      FROM
  9      (
 10          SELECT SUM(SUBTOTAL) AS PRODUCT_SALES
 11          FROM SAVANA_ORDER_DETAILS
 12          GROUP BY PRODUCT_ID
 13      )
 14  );

PRODUCT_ID TOTAL_SALES
---------- -----------
       201        2997
```

6. Customer Purchase Report

A detailed customer purchase report can combine Customer and Orders tables.

```
SQL> SELECT CUSTOMER_ID,
  2         COUNT(ORDER_ID) AS NUMBER_OF_ORDERS,
  3         SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE,
  4         AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  5  FROM SAVANA_ORDERS
  6  GROUP BY CUSTOMER_ID;

CUSTOMER_ID NUMBER_OF_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_VALUE
----------- ---------------- -------------- -------------------
        101                2          42997             21498.5
        102                1           1999                1999
        103                1           1598                1598
```

7. Advanced Product Sales Report

This report combines Product and Order_Details tables.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         COUNT(D.ORDER_ID) AS NUMBER_OF_ORDERS,
  4         SUM(D.QUANTITY) AS TOTAL_QUANTITY,
  5         SUM(D.SUBTOTAL) AS TOTAL_SALES,
  6         AVG(D.UNIT_PRICE) AS AVERAGE_PRICE
  7  FROM SHOP_PRODUCT P
  8  JOIN SAVANA_ORDER_DETAILS D
  9  ON P.PRODUCT_ID = D.PRODUCT_ID
 10  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY TOTAL_SALES AVERAGE_PRICE
---------------- -------------- ----------- -------------
       101
Wireless Headphones
               1              1        1799          1799

       201
T-Shirt
               2              3        2997           999

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY TOTAL_SALES AVERAGE_PRICE
---------------- -------------- ----------- -------------
```

8. Category-wise Advanced Sales Report

Category-wise sales can be generated by combining Category, Product, and Order_Details.

```
SQL> SELECT P.CATEGORY_ID,
  2         SUM(D.QUANTITY) AS TOTAL_QUANTITY,
  3         SUM(D.SUBTOTAL) AS TOTAL_SALES
  4  FROM SHOP_PRODUCT P
  5  JOIN SAVANA_ORDER_DETAILS D
  6  ON P.PRODUCT_ID = D.PRODUCT_ID
  7  GROUP BY P.CATEGORY_ID;

CATEGORY_ID TOTAL_QUANTITY TOTAL_SALES
----------- -------------- -----------
          1              1        1799
          2              3        2997
```